Part 2 report:
2.3. We chose SQLite because it is lightweight and requires no separate database server. Also, it integrates well with Python. We use two tables: the first one is documents. It stores article IDs, titles, and pagerank scores. The second one is Postings. It stores term, article IDs, and TF-IDF weights. The composite primary key is (term, article_id).
We considered PostgreSQL as an alternative. But it requires additional setup and server management. We also considered storing entire posting lists as serialized values, but using individual rows makes it easier to perform sql joins, filtering and ranking.
Performance:

- Index build time: 53s
- database load time: 45s
- storage size: 598M
- median query latency: 10.1ms
- p95 query latency: 32.9ms
- Max query latency: 41.1ms

```
(base) B99@Bs-MacBook cs511p2 % make test-search
Building the index and starting the API (full) ... PASS
  ready in 104s
Querying the API ... PASS
  15/15 queries correct: 15/15 points
Testing latency (p95 <= 200 ms) ... PASS
  LATENCY: median 10.1 ms, p95 32.9 ms, max 41.1 ms
----------------------------------
Result: 20/20 points
(base) B99@Bs-MacBook cs511p2 % grep -R -E "INDEX BUILD TIME|DATABASE LOAD TIME" out/
out/full/test_2_search.out:INDEX BUILD TIME: 53 seconds
out/full/test_2_search.out:DATABASE LOAD TIME: 45 seconds
(base) B99@Bs-MacBook cs511p2 % docker exec cs511p2-main-1 ls -lh /data/search.db
-rw-r--r-- 1 root root 598M Oct 31 00:53 /data/search.db
(base) B99@Bs-MacBook cs511p2 %
```

Its p95 latency of 32.9ms is below the required 200ms. The composite primary key supports efficient posting lookups, and precomputed TF-IDF weights reduce the amount of computation required at query time.